SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1– Pseudocode Design

Course Code:	**DSA0405**	**Course Title:**	**FUNDAMENTALS OF DATA SCIENCE**
CO Assessed:	**CO5 –**Pseudocode Design	**Assessment:**	**Assessment Tool 1–** Pseudocode Design
Weightage:	**30% of CIA**	**Total Marks:**	**100**
CO5 Statement:	To understand the concept of supervised and unsupervised methods in machine learning		
Student Name:	Jothika M	**Reg. No.:**	192324235
Section / Batch:	2023-B.Tech(AIDS)	**Date:**	25-08-26

Code of Conduct

I Jothika M (192324235) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Jothika M

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks
Understanding of Case	**25**	**Thorough understanding; clearly identifies all key issues**	**Good understanding with most issues identified**	**Basic understanding; some issues missed**	**Poor understanding; problem not identified correctly**	
Application of Concepts	**25**	**Effectively applies relevant concepts to analyze the case deeply**	**Applies appropriate concepts with minor gaps**	**Limited application of concepts**	**Fails to apply relevant concepts**	
Analysis	**20**	**In-depth analysis with strong reasoning and insights**	**Good analysis with logical reasoning**	**Basic analysis with limited depth**	**Weak or no analysis; lacks reasoning**	
Solution Design	**20**	**Innovative, feasible solutions with strong justification**	**Logical solutions with adequate justification**	**Basic solutions with weak justification**	**Incorrect or no solution provided**	
Clarity	**10**	**Clear, well-structured, and easy to understand**	**Organized and understandable presentation**	**Limited clarity and structure**	**Poor clarity; difficult to understand**	

1.Design pseudocode to build a supervised learning workflow that accepts a labelled dataset, separates features and target values, trains a model, and predicts the target for new data.

Problem Statement

The objective is to design a supervised learning workflow using a labelled dataset. The dataset should be separated into input features and target values. The data is divided into training and testing sets for model development. A suitable supervised learning model is trained using the training data. The trained model is then used to predict the target value for new data.

Algorithm

1. Start.
2. Load the labelled dataset.
3. Separate the features and target values.
4. Split the dataset into training and testing data.
5. Train a supervised learning model.
6. Input new data.
7. Predict the target value using the trained model.
8. Display the prediction.
9. Stop.

Pseudocode

BEGIN

LOAD labelled dataset

X ← Features

Y ← Target values

SPLIT X and Y into training_data and testing_data

MODEL ← Create supervised learning model

TRAIN MODEL using training_data
INPUT new_data
prediction ← MODEL.predict(new_data)
DISPLAY prediction
END

Output

Dataset loaded successfully
Features and target separated
Model trained successfully
New Data: [80, 75, 90]
Predicted Target: Pass

2. Design pseudocode for a k-Nearest Neighbors (kNN) classifier that calculates the distance between a test sample and all training samples, selects the k nearest samples, and predicts the class using majority voting.

Problem Statement

The objective is to design a k-Nearest Neighbors classifier for classifying a test sample. The distance between the test sample and every training sample must be calculated. The training samples are sorted according to their calculated distances. The k nearest samples are selected for classification. The final class is predicted using majority voting among the selected neighbors.

Algorithm

1. Start.
2. Input training data, labels, test sample, and k.

3. Calculate the distance between the test sample and each training sample.
4. Sort the samples based on distance.
5. Select the k nearest samples.
6. Count the class labels of the selected samples.
7. Select the class with the highest count.
8. Display the predicted class.
9. Stop.

Pseudocode

```
BEGIN
INPUT training_data
INPUT training_labels
INPUT test_sample
INPUT k
FOR each sample in training_data
   distance ← Calculate distance(sample, test_sample)
   STORE distance and label
END FOR
SORT samples by distance
nearest_samples ← First k samples
predicted_class ← Majority_Vote(nearest_samples)
DISPLAY predicted_class
END
```

Output

Test Sample: [6, 7]

k = 3

Nearest Classes:

Class A

Class B

Class A

Majority Class: Class A

Predicted Class: Class A

3. kNN Student Pass/Fail Classification - Design pseudocode to classify a new student as Pass or Fail using kNN based on attendance percentage, internal marks, and assignment scores.

Problem Statement

The objective is to classify a new student as Pass or Fail using the kNN algorithm. The student's attendance percentage, internal marks, and assignment score are used as input features. The distance between the new student and existing training students is calculated. The k nearest students are selected based on their distances. The student's result is predicted as Pass or Fail using majority voting.

Algorithm

1. Start.
2. Input training student records and their results.
3. Input attendance, internal marks, and assignment score of the new student.
4. Calculate the distance between the new student and each training student.
5. Sort the distances.
6. Select the k nearest students.
7. Count Pass and Fail results.
8. Select the result with the highest count.
9. Display the prediction.
10. Stop.

Pseudocode

```
BEGIN
INPUT training_student_data
INPUT training_labels
INPUT attendance
INPUT internal_marks
INPUT assignment_score
new_student ← [attendance, internal_marks, assignment_score]
INPUT k
FOR each student in training_student_data
   distance ← SQRT(
      (attendance - student.attendance)^2 +
      (internal_marks - student.internal_marks)^2 +
      (assignment_score - student.assignment_score)^2
   )
   STORE distance and result
END FOR
SORT distances in ascending order
nearest_students ← First k students
pass_count ← COUNT Pass in nearest_students
fail_count ← COUNT Fail in nearest_students
IF pass_count > fail_count THEN
   prediction ← "PASS"
ELSE
   prediction ← "FAIL"
END IF
DISPLAY prediction
END
```

Output

Attendance = 85%

Internal Marks = 78

Assignment Score = 90

k = 3

Nearest Students:

Pass

Pass

Fail

Pass Count = 2

Fail Count = 1

Predicted Result: PASS

4. Selecting the Appropriate Value of k- Design pseudocode for selecting the appropriate value of k in a kNN classifier by evaluating different k values on validation data.

Problem Statement

The objective is to determine the most suitable value of k for a kNN classifier. Different values of k are selected and tested using validation data. The classifier is evaluated for each selected value of k. The accuracy obtained for every value is calculated and compared. The value of k with the highest validation accuracy is selected as the appropriate value.

Algorithm

1. Start.
2. Load the labelled dataset.
3. Divide the dataset into training and validation data.

4. Select different values of k.
5. Classify the validation data for each k.
6. Calculate the accuracy for each value.
7. Compare the accuracies.
8. Select the k with the highest accuracy.
9. Display the best k and accuracy.
10. Stop.

Pseudocode

```
BEGIN
LOAD labelled dataset
SPLIT dataset into training_data and validation_data
K_VALUES ← [1, 3, 5, 7, 9]
best_k ← 0
best_accuracy ← 0
FOR each k in K_VALUES
   predictions ← EMPTY
   FOR each sample in validation_data
      distances ← Calculate distances
                  between sample and training_data
      SORT distances
      nearest_samples ← First k samples
      prediction ← Majority_Vote(nearest_samples)
      ADD prediction to predictions
   END FOR
   accuracy ← Calculate_Accuracy(predictions)
   DISPLAY k, accuracy
   IF accuracy > best_accuracy THEN
```

```
        best_accuracy ← accuracy
        best_k ← k
    END IF
END FOR
DISPLAY best_k
DISPLAY best_accuracy
END
```

Output

Validation Results:

k = 1 → Accuracy = 85%

k = 3 → Accuracy = 90%

k = 5 → Accuracy = 94%

k = 7 → Accuracy = 92%

k = 9 → Accuracy = 89%

Best k = 5

Best Validation Accuracy = 94%

5. Decision Tree Classifier - Design pseudocode to construct a Decision Tree classifier by selecting the best feature for splitting the training dataset and recursively creating child nodes.

Problem Statement

The objective is to design a Decision Tree classifier using a labelled training dataset. The available features are evaluated to identify the best feature for splitting the dataset. The dataset is divided into smaller subsets based on the selected feature. The splitting process is repeated recursively for each child node. The process continues until suitable leaf nodes containing the predicted classes are created.

Algorithm

1. Start.
2. Load the labelled training dataset.
3. Check whether all samples belong to the same class.
4. If yes, create a leaf node.
5. Otherwise, calculate the quality of each feature split.
6. Select the best feature for splitting.
7. Divide the dataset using the selected feature.
8. Recursively create child nodes.
9. Continue until leaf nodes are created.
10. Display the Decision Tree.
11. Stop.

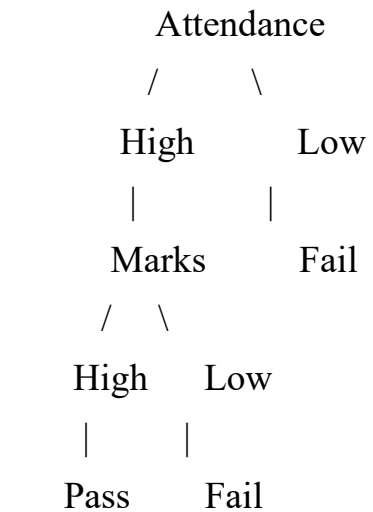
Pseudocode

```
BEGIN
FUNCTION BUILD_TREE(dataset, features)
   IF all samples belong to same class THEN
      RETURN Leaf(class)
   END IF
   IF features is empty THEN
      RETURN Leaf(Majority_Class(dataset))
   END IF
   best_feature ← Select feature with best split
   NODE ← Create Decision Tree Node
   NODE.feature ← best_feature
   FOR each value of best_feature
      subset ← Select samples with this value
```

```
  IF subset is empty THEN
        child ← Leaf(Majority_Class(dataset))
     ELSE
        remaining_features ← features - best_feature
        child ← BUILD_TREE(subset, remaining_features)
     END IF
     ADD child to NODE
   END FOR
   RETURN NODE
END FUNCTION
LOAD training_dataset
features ← Dataset features
tree ← BUILD_TREE(training_dataset, features)
DISPLAY tree
END
```

Output

Decision Tree Constructed Successfully



New Student:

Attendance = High

Marks = High

Predicted Class: PASS